

12.5 Vector Store Databases

To avoid having to recalculate the vector embeddings every time the application is started, we'll save the index in a database. The task of the vector database is to store the numerical representation of unstructured data (the vectors with their dimensions) and to search for them using various algorithms. The implementations are divided into those in which external libraries perform this search in an existing (relational or NoSQL) database, those in which plug-ins are used in the database, and *genuine* vector databases.

Real vector databases are on a par with SQL and NoSQL databases. While SQL databases force the source data into a strong structure (through rigid tables with many data types), NoSQL structures the data only roughly through key-value pairs with data types such as numbers, strings, or dates. With vector databases, the source data is stored unstructured, and only the calculated vectors are used in the search.

A prominent representative of genuine vector databases is Milvus. The database is available under the free Apache 2 license and can be used very easily in different scenarios: The Lite version is ideal for *prototyping* or on devices with very low-end hardware. For production deployments, *Milvus Standalone* can be started very easily in the Docker container. If you think big, you can also start Milvus in a Kubernetes cluster in a cloud-native architecture. In addition, you can also rent Milvus as a ready-made cloud solution from Zilliz, the company behind the development.

We tried out both the Lite version and the standalone version. Both variants were convincing in our very small dataset. The Docker setup with a `compose.yaml` file can be started practically at the touch of a button. The API calls don't differ whether you use Lite/standalone or the cloud version of Milvus. This allows you to turn your small test project into a scalable solution in just a few seconds.

We use the standalone version of Milvus in the Docker setup with compose for our RAG project. If you've installed the latest version of Docker and the compose plug-in, all you need to do is download the configuration file provided on GitHub and start the setup via `docker compose up -d`:

<https://github.com/milvus-io/milvus/releases/download/v2.4.6/milvus-standalone-docker-compose.yml>

In addition to the database, two other services are started: the *etcd* configuration service and the *MinIO* object store. The database becomes available shortly afterward on port 19530 without authentication.

Zilliz has also developed a graphical user interface (GUI) for managing the database as open-source software showing the indexed PDF documents and web pages. On the left of the screen are three collections, each created with different embedding models (see [Figure 12.3](#)).

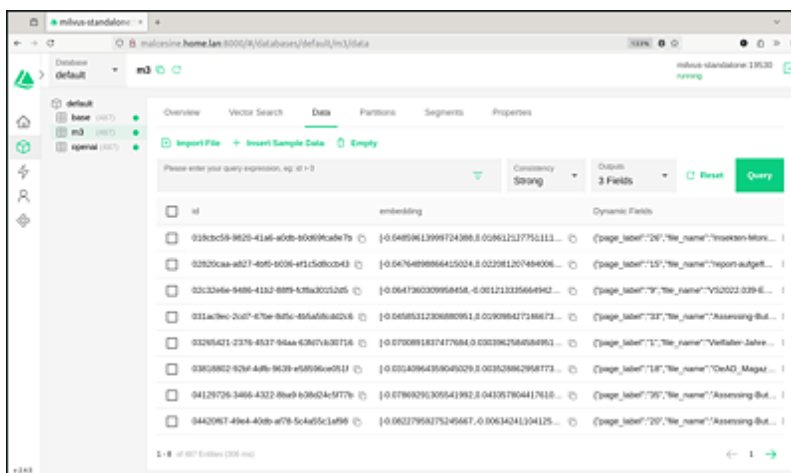


Figure 12.3 Attu Displaying the Milvus Vector Database

Attu can easily be added as an additional container to the Docker Compose setup. Simply add the following lines to the `services` section of the downloaded Compose file:

```
attu:
  image: zilliz/attu:v2.4
  environment:
    MILVUS_URL: milvus-standalone:19530
  ports:
    - "8000:3000"
```

After starting the container using `docker compose up -d`, you can access the web interface at <http://localhost:8000> or the host name on which you started the docker compose setup. We found the insight into the internal data structure of the vector database very fascinating.

To use the Milvus database for our sample project, we include a `StorageContext LlamaIndex` when creating the `VectorStoreIndex`:

```
from llama_index.vector_stores.milvus import
MilvusVectorStore
from llama_index.embeddings.ollama import
OllamaEmbedding
from llama_index.core import StorageContext
vector_store = MilvusVectorStore(uri="http://localhost:19530",
                                dim=1024,
                                overwrite=True)
storage_context = StorageContext.from_defaults(
    vector_store=vector_store
)
index = VectorStoreIndex.from_documents(
    documents=documents,
    transformations=[
        SentenceSplitter(),
        TitleExtractor(),
        OllamaEmbedding(model_name="mxbai-embed-
large"),
    ],
    storage_context=storage_context,
    show_progress=True,
)
```

In addition to `uri`, which refers to the database previously started in Docker, the dimension of the vectors must also be specified and must then be adapted to the embedding model used. The `mxbai-embed-large` model used here uses vectors with 1,024 dimensions. To avoid duplicates, the `overwrite=True` parameter deletes the database before the index is created. The application is now ready, and we can try out the first queries in natural language using the indexed content.

